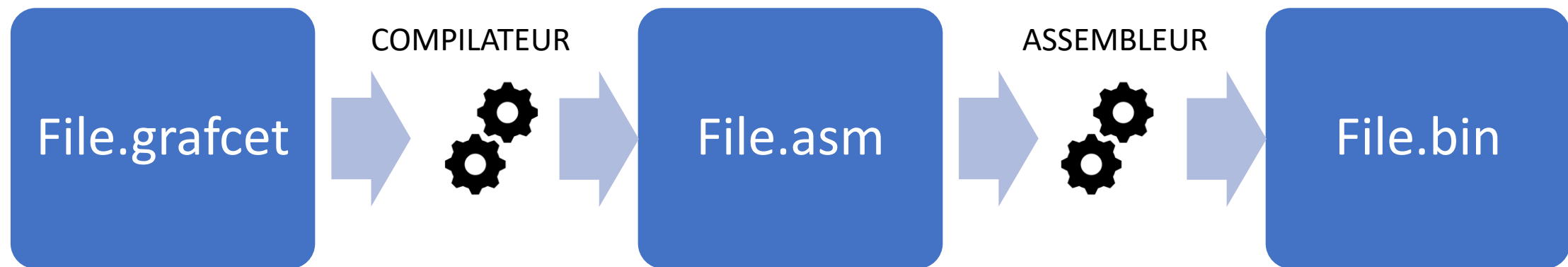
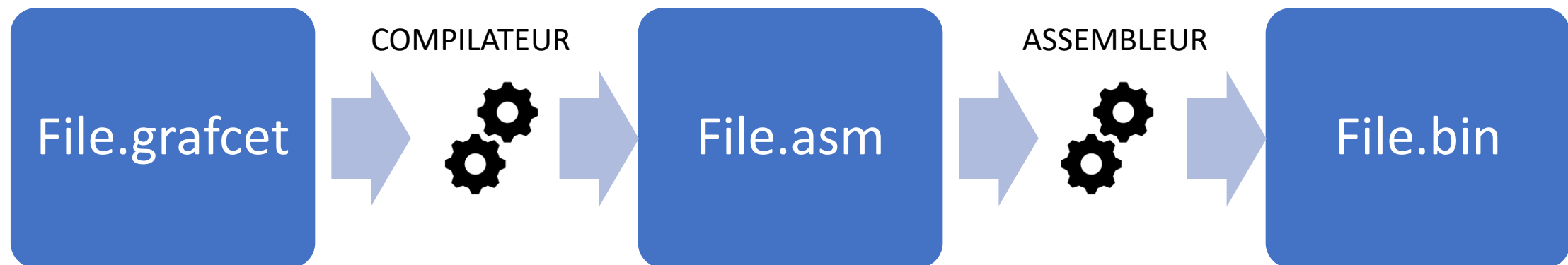


Projet Info EI2I3 – 3/3/2017

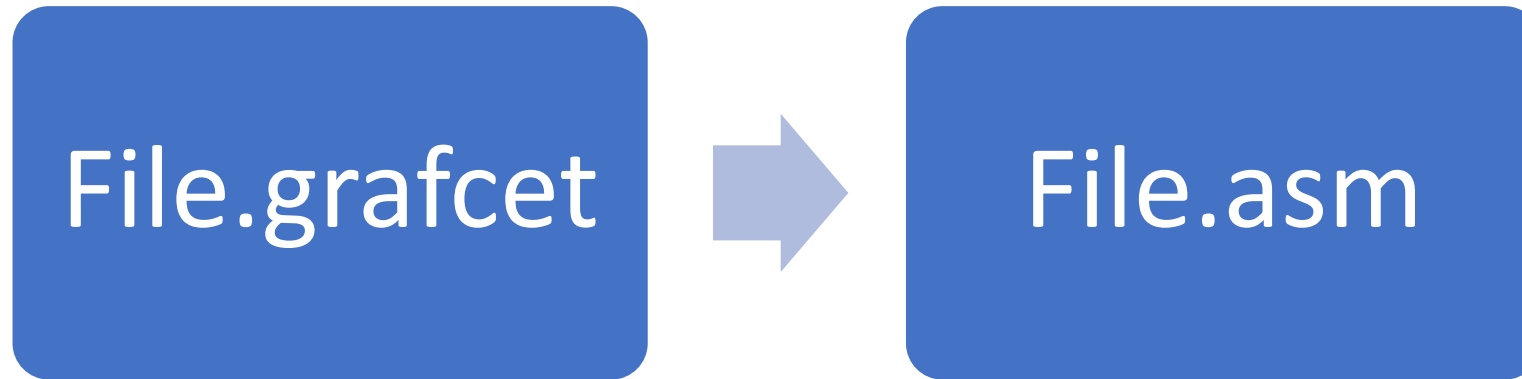
1. Rappel
2. Le compilateur
3. L'assembleur
4. Le projet





active step e0(led0=0)	#franchissement de la transition t0	269
step e1(led0=0)	push 14	0:101
step e2(led0=0)	pushi 1	1:0
step e3(led0=1)	eq	2:102
	push 1	3:13
transition t0({e0},{e1},but0'rise.(ana0<128))	push 6	4:101
transition t1({e1},{e2},but0'rise.(ana0>128))	pushi 128	5:0
transition t2({e2},{e3},but0'rise.(ana0<128))	ls	6:102
transition t3({e3},{e0},but1'rise)	and	7:22
	and	8:101
	jf next0000	9:0
	pushi 1	10:102
	pop 23	11:26
	pushi 1	..
	pop 26	.
	next0000:	
	..	
	.	

Compilateur



Compilateur:

- ✓ Interprétation du grafcet
- Analyse lexical
 - Lex
- Analyse syntaxique
 - Bison
- Construction et exploitation d'un arbre syntaxique (AST)

Exemple de compilation C vers ASM

```
// Franchissement de la condition t0
if ((e0==1) && ((risebut0==1) && (an0<128))){
    appel_e1=1;
    reponse_e0=1;
}
```

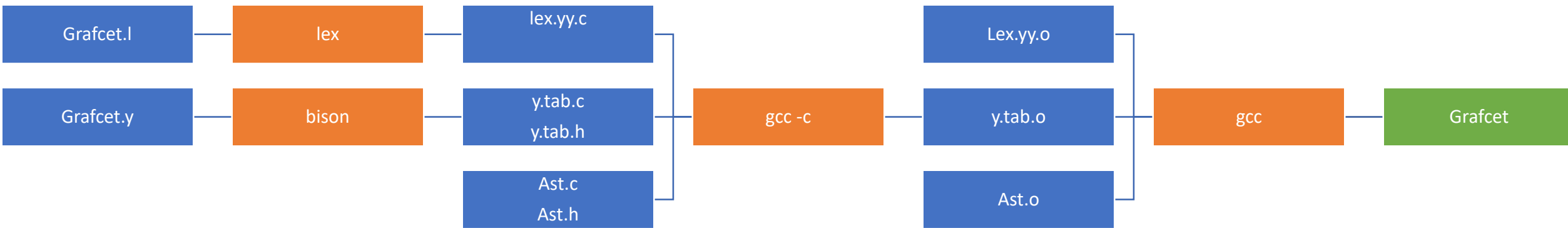
Exemple de compilation C vers ASM

```
// Franchissement de la condition t0
if ((e0==1) && ((risebut0==1) && (an0<128))){
    appel_e1=1;
    reponse_e0=1;
}
```



```
# 001 : but0'rise
# 006 : an0
# 014 : e0
# 023 : appel_e1
# 026 : reponse_e0
#franchissement de la transition t0
    push    14
    pushi   1
    eq
    push    1
    pushi   1
    eq
    push    6
    pushi   128
    ls
    and
    and
    jf      next0000
    pushi   1
    pop     23
    pushi   1
    pop     26
next0000:
..
.
```

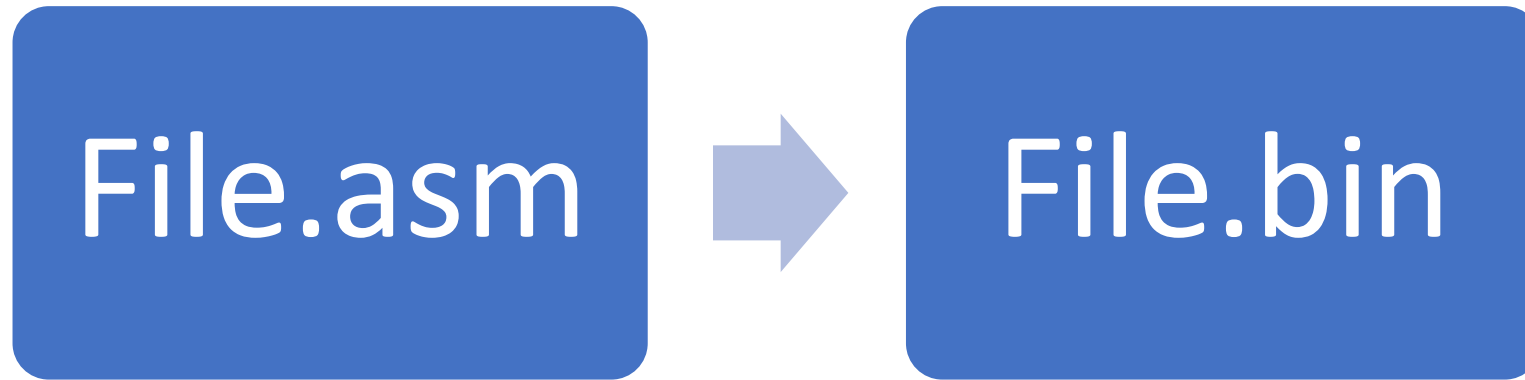
Construction du compilateur grafcet.out



Utilisation en console

```
remi@VirtualBox:~/.$./grafcet safe.grafcet safe.asm
```

Assembleur



Assembleur:

1ere phase d'assemblage:

- Génère un tableau de OP_CODE
 - Analyse et décode les instructions (parseur)
 - Construit la liste des références indéfinies

2eme phase d'assemblage:

- Complète le tableau de OP_CODE
 - Résoud les références indéfinies

Assembleur

```
# 0 : a
# 1 : b
pushi    0
push     0
eq
jf      toto
pushi    1
pop      1
jp      toto1
Toto:
pushi    2
pop      1
Toto1:
..
.
```

Assembleur

1ere phase

	# 0 : a	0:100
	# 1 : b	1:0
	pushi 0	1:101
	push 0	3:0
	eq	4:12
	jf toto	5:200
	pushi 1	6:-1
	pop 1	7:100
	jp toto1	8:1
Toto:		9:102
	pushi 2	10:1
	pop 1	11:201
Toto1:		12:-1
..		13:100
.		14:2
		15:102
		16:1
		17:....



Référence non résolu:

Référence à Toto à l'index 6

Référence à Toto1 à l'index 12

Etiquette:

Etiquette Toto à l'index 13

Etiquette Toto1 à l'index 17

Assembleur

1ere phase

2e phase

	# 0 : a	0:100
	# 1 : b	1:0
	pushi 0	1:101
	push 0	3:0
	eq	4:12
	jf toto	5:200
	pushi 1	6:-1
	pop 1	7:100
	jp toto1	8:1
Toto:		9:102
	pushi 2	10:1
	pop 1	11:201
Toto1:		12:-1
..		13:100
.		14:2
		15:102
		16:1
		17:....

Référence non résolu:

Référence à Toto à l'index 6

Référence à Toto1 à l'index 12

Etiquette:

Etiquette Toto à l'index 13

Etiquette Toto1 à l'index 17

0:100
1:0
1:101
3:0
4:12
5:200
6:13
7:100
8:1
9:102
10:1
11:201
12:17
13:100
14:2
15:102
16:1
17:....

Assembleur

Utilisation en console

```
remi@VirtualBox:~/.$ ./asm safe.asm safe.bin
```

Asm.c

Utilisation en console

```
remi@VirtualBox:~/.$./asm safe.asm safe.bin
```

```
int main(int argc, char **argv){  
    // Ouverture du fichier  
    ...  
    // Construction du dictionnaire  
d'instruction  
    addInstructionName(...)  
    // Parseur phase 1  
    parseAsm(fin);  
    // Parseur phase 2  
    resolveReferences();  
    // Génération du code  
    generateBinary(fout);  
    return 0;  
}
```

```
void parseAsm(FILE *fin)
```

Cette fonction parcourt les lignes du fichier source assembleur tant qu'elle n'a pas rencontré de ligne avec "end". Pour chaque ligne il y a 3 cas de figure:

- si la ligne de texte contient : il s'agit d'une étiquette, on appelle la fonction

```
addLabel
```

- si on trouve #, c'est un commentaire
- sinon, il s'agit d'une instruction et on appelle la fonction

```
decodeInstruction
```

```
void addLabel(char *labelname, int addr)
```

Appel:

```
addLabel(line, currentInst);
```

line: ligne courante

currentInst: Variable globale indiquant la position de l'instruction courante, incrémentée après l'ajout de chaque instruction.

Complète le tableau de structure contenant les labels (variable globale)

```
struct label {  
    char *label;  
    int addr;  
} tabLabels[MAX_LABELS_SIZE];
```

Incrémente le compteur de label (variable globale)

```
int currentLabel;
```

```
void addLabel(char *labelname,int addr)
```

Fonctions utiles:

```
fgets(line,100,fin);
```

```
strstr(line,"end");
```



```
void decodeInstruction(char *line)
```

Appel:

```
decodeInstruction(line) ;
```

line: ligne courante

Cette fonction fait le gros du travail:

- Identifie le type de l'instruction
- Fait le décodage supplémentaire
- Appel `addCode` pour remplir `codeSegment` (tableau global `unsigned int`)
- Ajoute des références si l'instruction décodée fait référence à une étiquette encore inconnue (tableau global)

```
struct ref {
```

```
    char *label;
```

```
    int addrInCode;
```

```
} tabReferences[MAX_LABELS_SIZE];
```

```
int currentRef;
```

```
void decodeInstruction(char *line)
```

Cette fonction va comparer chaque instruction trouvée à un dictionnaire d'instruction préalablement construit par le programmeur. Ce dictionnaire est en fait un tableau de structure.

```
struct instructionName {  
    char *name;           // Le nom de l'instruction  
    int opcod;            // son codop, extrait de vm_codops.h  
    int type;             // son type  
    char *format;         // son format  
    int nbops;            // le nombre d'operandes de l'inst  
} tabInstructionNames[MAX_IDENTS_SIZE];
```

Une instruction est définie par son nom, son codop, son type, son format et son nombre d'arguments.
On distingue 3 types d'instructions

Construction du dictionnaire

Instructions de Type 0:

Elles ne demandent pas de décodage d'operandes supplementaires. Leur chaine de format est "" et leur nombre d'argument est 0

Construction du dictionnaire

Instructions de Type 0:

Elles ne demandent pas de décodage d'operands supplémentaires. Leur chaîne de format est "" et leur nombre d'argument est 0

```
addInstructionName("add", OP_ADD, 0, "", 0);  
addInstructionName("sub", OP_SUB, 0, "", 0);  
addInstructionName("mult", OP_MULT, 0, "", 0);  
addInstructionName("div", OP_DIV, 0, "", 0);  
addInstructionName("neg", OP_NEG, 0, "", 0);  
addInstructionName("and", OP_AND, 0, "", 0);  
addInstructionName("or", OP_OR, 0, "", 0);  
addInstructionName("not", OP_NOT, 0, "", 0);  
addInstructionName("eq", OP_EQ, 0, "", 0);  
addInstructionName("ls", OP_LS, 0, "", 0);  
addInstructionName("gt", OP_GT, 0, "", 0);  
addInstructionName("halt", OP_HALT, 0, "", 0);
```

Construction du dictionnaire

Instructions de Type 1:

Le décodage d'un entier est nécessaire. La chaîne de format est donc "%s %d". Le %s représente l'instruction et le %d l'opérande. Il y a une opérande pour ce genre d'instruction

Construction du dictionnaire

Instructions de Type 1:

Le décodage d'un entier est nécessaire. La chaîne de format est donc "%s %d". Le %s représente l'instruction et le %d l'opérande. Il y a un opérande pour ce genre d'instruction

```
addInstructionName("push", OP_PUSH, 1, "%s %d", 1);  
addInstructionName("pop", OP_POP, 1, "%s %d", 1);  
addInstructionName("pushi", OP_PUSH_I, 1, "%s %d", 1);
```

Construction du dictionnaire

Instructions de Type 3:

le decodage d'une chaine de caracteres representant une etiquette est necessaire. La chaine de format est donc "%s %s" et le deuxieme %s represente l'etiquette. Il y a une operande pour ce genre d'instructions

Construction du dictionnaire

Instructions de Type 3:

le decodage d'une chaine de caracteres representant une etiquette est necessaire. La chaine de format est donc "%s %s" et le deuxieme %s represente l'etiquette. Il y a une operande pour ce genre d'instructions

```
addInstructionName("jp", OP_JP, 3, "%s %s", 1);
```

```
addInstructionName("jf", OP_JF, 3, "%s %s", 1);
```

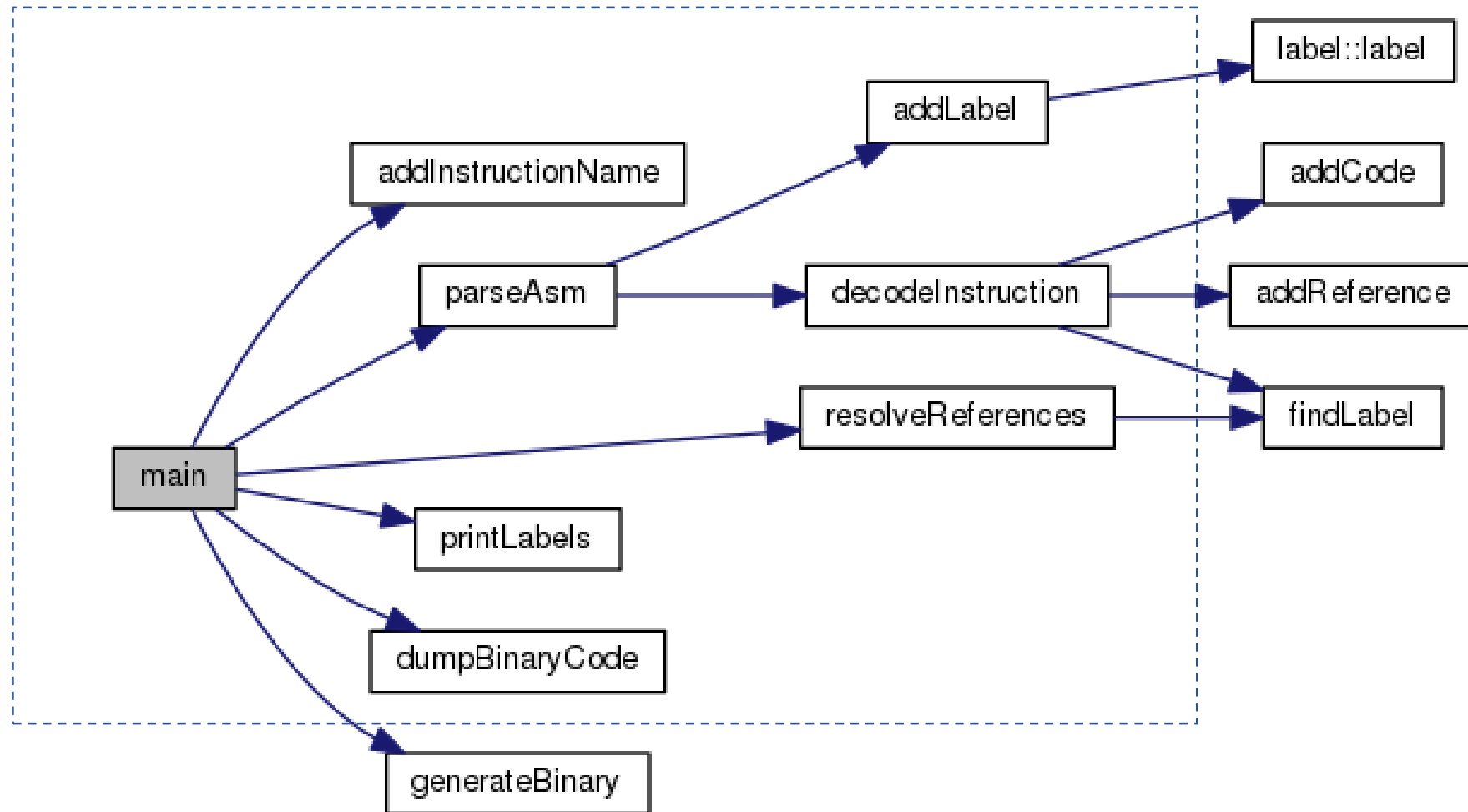

void dumpBinaryCode();

Fonction de désassemblage du code (fonction de vérification)

Elle permet de générer le code assembleur correspondant aux OP_CODE stockés dans `codeSegment`

Sa sortie peut être en console ou dans un fichier.

Assembleur



Projet

- Jalon
 - GraphcetInterpreter, VM et Assembleur opérationnel pour le jeudi 23/3/2017
- Documentation du code
 - Doxygen
- Gestion de version (travail en équipe)
 - GitHub
- A terme, execution du ByteCode par une VM sur MCU
 - Communication série